

CIS 371 Web Application Programming

TypeScript VI



Lecturer: **Dr. Haoyu Li**



Recall

- Type Alias vs. Interface vs. Class, Inheritance
- Generics in TypeScript

Practice 1: Interface

// 1. Create an interface Person with properties name: string and age: number.

```
interface Person {  
  name: string;  
  age: number;  
}
```

// 2. Create another interface Student that extends Person and adds studentId: number.

```
interface Student extends Person {  
  studentId: number;  
}
```

// 3. Write a function printStudent(student: Student): void that prints name, age, and studentId.

```
function printStudent(student: Student): void {  
  console.log(`Name: ${student.name}`);  
  console.log(`Age: ${student.age}`);  
  console.log(`Student ID: ${student.studentId}`);  
}
```

// 4. Call the function with a student object.

```
const student1: Student = {  
  name: "Alice",  
  age: 21,  
  studentId: 1001  
};
```

```
printStudent(student1);
```

Practice 2: Generic Type

```
// Write a generic function getFirst that accepts an array of any type and  
returns the first element, if present.  
const getFirst = <T>(arr: T[]): T | undefined => {  
  return arr.length > 0 ? arr[0] : undefined;  
};
```

Using TypeScript in Browser

Browser Debugger (Chrome)

Debugger Controls
(step over, step into, ...)

Debugging breakpoints

The screenshot displays the Chrome DevTools interface with the **Sources** panel active. The file `history_cluster...mojom-webui.js` is open, and a breakpoint is set at line 9. The **Debugger Controls** (step over, step into, etc.) are visible in the top right. The **Variable Inspector** on the right shows the current scope and variables.

Debugger Controls (step over, step into, ...)

Debugging breakpoints

Variable Inspector

Paused on breakpoint

- Threads
- Watch
- Breakpoints
 - ☐ Pause on uncaught exceptions
 - ☐ Pause on caught exceptions
- history_clusters_layout_type.mojom-webui.js
 - ☒ LayoutType[LayoutType["kNone"] = 0] = "kNone"; 9
 - ☒ LayoutType[LayoutType["MIN_VALUE"] = 0] = "MIN_VALUE"; 15
- Scope
 - Local
 - this: undefined
 - LayoutType: {}
 - Module
 - LayoutType: {}
 - LayoutTypeSpec: {\$: {-}}
 - mojo: {internal: {-}, interfaceControl: {-}, pipeControl: {-}}
 - Global
- Call Stack
 - (anonymous) history_cluster...jom-webui.js:9
 - (anonymous) history_cluster...om-webui.js:17
 - XHR/fetch Breakpoints
 - DOM Breakpoints
 - Global Listeners
 - Event Listener Breakpoints
 - CSP Violation Breakpoints

How does the browser load and run your code?

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Document</title>
  </head>
  <body>
    <h1>Hello</h1>
    <script>
      console.log(document.body.innerHTML);
    </script>
    <p>World!</p>
  </body>
</html>
```



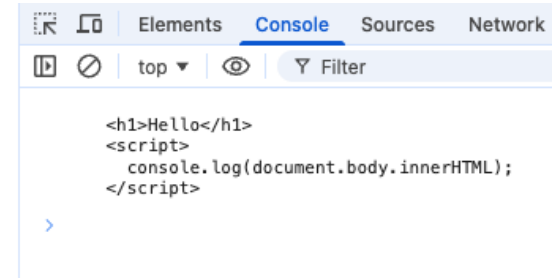
What will be printed?

How does the browser load and run your code?

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Document</title>
  </head>
  <body>
    <h1>Hello</h1>
    <script>
      console.log(document.body.innerHTML);
    </script>
    <p>World!</p>
  </body>
</html>
```

Hello

World!



Why?

How does the browser load and run your code?

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <script src="code1.js"></script>
  </head>
  <body>
    <!--    other HTML contents go here    -->
    <script src="code2.js"></script>
  </body>
</html>
```

Scripts that do not modify page contents are placed in <head>

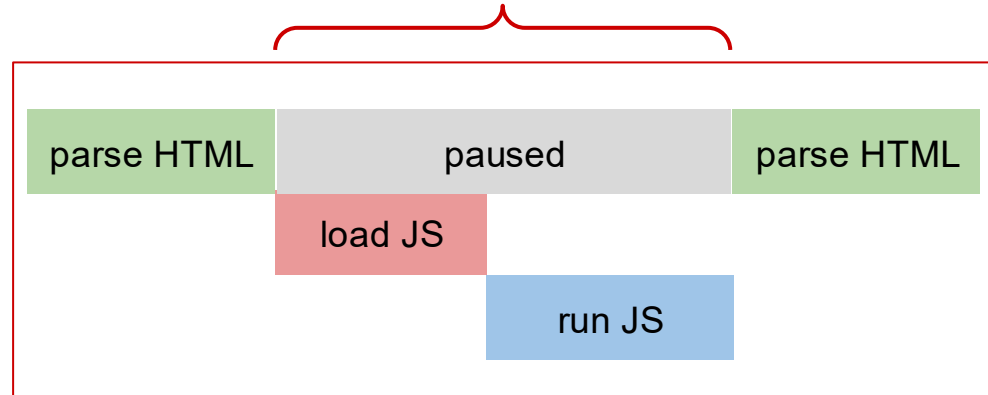
Scripts that do are placed towards the end of <body>

Script: Loading & Running

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <title>
  </head>
  <body>
    <!--   some HTML here   -->
    <script src="....js"></script>
    <!--   more HTML here   -->
  </body>
</html>
```

<script>

HTML Parsing suspended



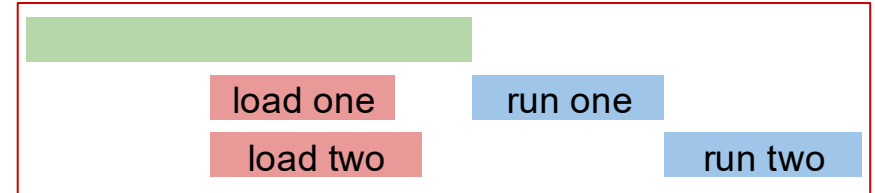
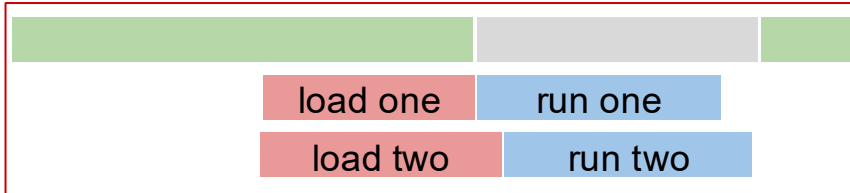
Defer vs. Async

```
<html>  
  <body>  
    <!-- some HTML here -->  
    <script src="one" async></script>  
    <script src="two" async></script>  
    <!-- more HTML here -->  
  </body>  
</html>
```

`<script async>`

```
<html>  
  <body>  
    <!-- some HTML here -->  
    <script src="one" defer></script>  
    <script src="two" defer></script>  
    <!-- more HTML here -->  
  </body>  
</html>
```

`<script defer>`



When should I use what?

Typically, you want to use **async** where possible, then **defer**, then **normal** mode. Here are some general rules to follow:

- If the script is modular and does not rely on any other script, then use **async**.
- If the script relies upon or is relied upon by another script, then use **defer**.
- If the script is small and is relied upon by an **async** script, then use an inline script with **normal** mode (no extra attribute) placed above the **async** scripts.

async vs defer attributes

Browser predefined classes

- Classes associated with individual HTML tags

Tag	Class
<a>	<u>HTMLAnchorElement</u>
<body>	<u>HTMLBodyElement</u>
<button>	<u>HTMLButtonElement</u>
	<u>HTMLImageElement</u>
<p>	<u>HTMLParagraphElement</u>
<u><i>and many more ...</i></u>	

- Other classes: `AudioBuffer`, `Bluetooth`, `ByteString`, `Promise`, `Request`,...

Browser (Predefined) Objects

- Frequently used
 - screen: the computer screen occupied by the browser
 - document: the current HTML document that hosts the script
 - Provides functions for manipulating the DOM tree
 - window: the current window where the HTML doc is rendered
- Less frequently used
 - history: page visit history stack
 - localStorage: the browser persistent storage
 - location: the browser input box
 - and many more ...

```
for (const z in window) {  
  if (typeof window[z] === "object") {  
    console.log(z);  
  }  
}  
  
window VM81:3  
self VM81:3  
document VM81:3  
location VM81:3  
customElements VM81:3  
history VM81:3
```

```
for (const z in window) {  
  if (typeof window[z] === "object") {  
    console.log(z);  
  }  
}
```

Try this yourself

Browser window predefined functions

- `alert()`: show an info dialog on the browser
- `addEventListener()`: setup event listeners
- `confirm()`: show a yes/no dialog
- `prompt()`: show an input dialog
- `setInterval()`, `setTimeout()`: start a timer
- `clearInterval()`, `clearTimeout()`: reset existing timer
- ...
- and many more ...

```
for (const z in window) {  
    if (typeof window[z] === "function") {  
        console.log(z);  
    }  
}
```

Try this yourself

```
for (const z in window) {  
    if (typeof window[z] === "function") {  
        console.log(z);  
    }  
}
```

alert	VM198:3
atob	VM198:3
blur	VM198:3
btoa	VM198:3
cancelAnimationFrame	VM198:3

Complete documentations: [Web Windows API](#) (MDN: Mozilla Dev Network)

HTML Document CRUD methods/functions

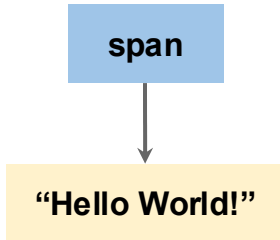
Create	<code>document.createElement()</code> , <code>document.createTextNode()</code>
Read	<code>____.getElementById()</code> // SINGULAR <code>____.getElementsByTagName()</code> // PLURAL <code>____.getElementsByClassName()</code> // PLURAL <code>____.querySelector()</code> // SINGULAR: search by CSS <code>selectors ____.querySelectorAll()</code> // PLURAL: search by CSS <code>selectors</code>
Update	<code>____.appendChild()</code>
Delete	<code>____.removeChild()</code>
<u>and many more ...</u>	

```
for (const z in document) {  
    if (typeof document[z] === "function") {  
        console.log(z);  
    }  
}
```

Try this yourself

Create Text Nodes

```
<span>Hello world!</span>
```

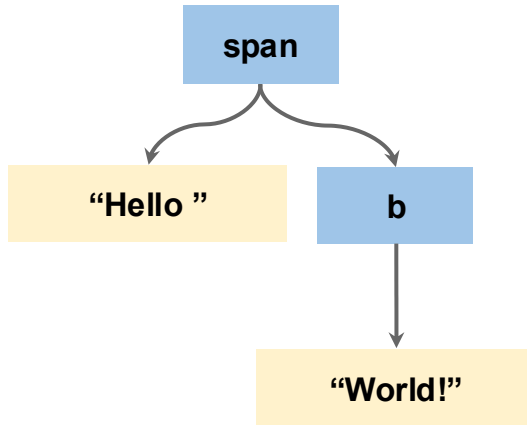


```
// Option 1
const spanParent = document.createElement("span");
const hello = document.createTextNode("Hello World");
spanParent.appendChild(hello);
```

```
// Option 2
const spanParent = document.createElement("span");
spanParent.innerText = "Hello World";
```

Add Multiple Children

```
<span>Hello <b>world!</b></span>
```



```
const spanTop = document.createElement("span");
const txt1 = document.createTextNode("Hello");
spanTop.appendChild(txt1);

const bChild = document.createElement("b");
bChild.innerText = "World";
spanTop.appendChild(bChild);
```

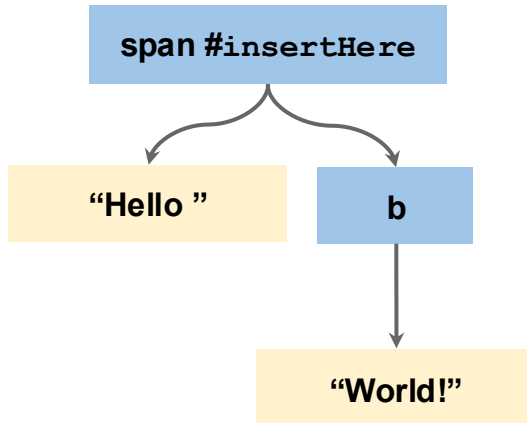
Insert Contents into Existing DOM

before

```
<span id="insertHere"> </span>
```

after

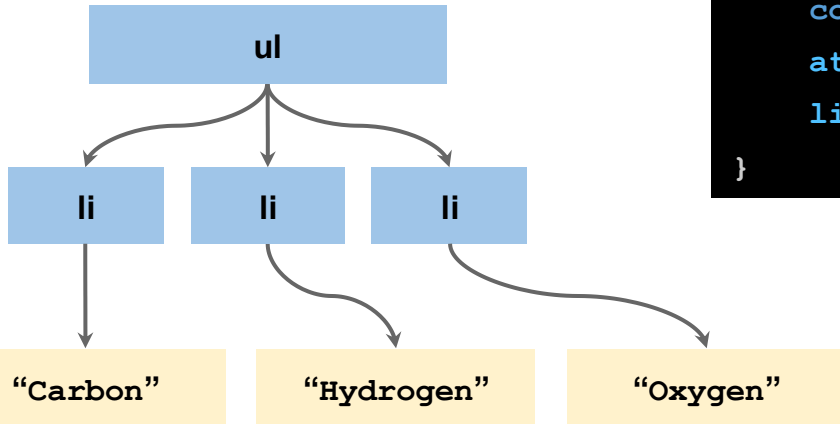
```
<span id="insertHere">  
  Hello <b>world!<b>  
</span>
```



```
const spanTop = document.getElementById("insertHere");  
const txt1 = document.createTextNode("Hello");  
spanTop.appendChild(txt1);  
  
const bChild = document.createElement("b");  
bChild.innerText = "World";  
spanTop.appendChild(bChild);
```

Add Multiple Children from Array

```
<ul>
  <li>Carbon</li>
  <li>Hydrogen</li>
  <li>Oxygen</li>
</ul>
```



```
const atoms = ["Carbon", "Hydrogen", "Oxygen"]
const listTop = document.createElement("ul");

for (let a of atoms) {
  const atm = document.createElement("li");
  atm.innerText = a;
  listTop.appendChild(atm);
}
```

Setting attributes

```
<a id="intro" class="deepIndent noAds" href="http://go.org">  
    Some text here  
</a>
```

```
const sample = document.createElement("a");  
sample.innerText = "Some text here";  
sample.id = "intro"  
sample.classList.add("deepIndent");  
sample.classList.add("noAds");  
sample.setAttribute("href", "http://go.org");
```

```
const sample = document.createElement("a");  
sample.innerText = "Some text here";  
sample.setAttribute("id", "intro");  
sample.setAttribute("class", "deepIndent noAds");  
sample.setAttribute("href", "http://go.org");
```

querySelector(): select ONE element

```
<body>
  <p class="title">Ice Cream Flavor:</p>
  <ul>
    <li>Death by Chocolate</li>
    <li>Mint Chocolate Chip</li>
    <li>Strawberry</li>
  </ul>
  <script src="ice.ts">
</body>
```

Ice Cream Flavors:

- Too much Chocolate
- Mint Chocolate Chip
- Strawberry

```
const item:Element = document.querySelector("ul > li");
#the first one will be returned
item.textContent = "Too much Chocolate";
```

querySelectorAll(): select MULTIPLE elements

```
<body>
  <p class="title">Ice Cream Flavor:</p>
  <ul>
    <li>Death by Chocolate</li>
    <li>Mint Chocolate Chip</li>
    <li>Strawberry</li>
  </ul>
  <script src="ice.ts">
</body>
```

Ice Cream Flavors:

- Death by Chocolate (on sale)
- Mint Chocolate Chip (on sale)
- Strawberry

```
let items:NodeListOf<Element>;
items = document.querySelectorAll("ul > li");
for (let flav of items) {
  if (flav.textContent.includes("Chocolate")){
    flav.textContent = flav.textContent + " (on sale)";
  }
}
```

CSS Selector and querySelector(All)

```
const q1 = document.querySelector("h2 + p");
q1.classList.add("red"); // Affect "First paragraph"
const q2 = document.querySelector("h2 ~ ol > li:first-child");
q2.classList.add("red"); // Affect "Strawberry"
const q3 = document.querySelector("li:last-child");
q3.classList.add("red"); // Affect "Barry Manilow"
```

```
<body>
  <h2>Some heading</h2>
  <p>First paragraph</p>
  <ol>
    <li class="fruit">Strawberry</li>
    <li class="device">Raspberry Pi</li>
    <li class="singer">Barry Manilow</li>
  </ol>
  <p>Second paragraph</p>
</body>
```

```
const pars = document.querySelectorAll("h2 ~ p");
for (let x of pars) {
  // Apply to "First paragraph" and "Second paragraph"
  x.setAttribute("__", "__");
}
const who = document.querySelectorAll("ol > li.singer");
for (let x of who) {
  // Apply to "Barry Manilow"
}
```


Using Timer

```
<body>
  <p>Ice Cream Flavor:</p>
  <ul>
    <li>Death by Chocolate</li>
    <li>Mint Chocolate Chip</li>
    <li>Strawberry</li>
    <li>Bluemoon</li>
  </ul>
  <script src="ice.ts">
</body>
```

```
function choco() {
  const item:Element = document.querySelector("ul > li");
  item.textContent = "Too much Chocolate";
}
setTimeout(choco, 2000);
```

setTimeout(someFunc, delayInMillisec)

Ice Cream Flavors:

- Too much Chocolate
- Mint Chocolate Chip
- Strawberry
- BlueMoon

2 seconds later

JavaScript Events

Source of Event	Events
Window	onload, onresize, onunload, ...
Document	onkeydown, onkeyup, onmousedown, onmouseup, onmouseenter, onmouseleave, ...
Input field	onblur, onfocus, onchange,
Button	onclick, ondblclick
Complete Reference: Event APIs	

Setting Up Event Handlers

- Which Event?
- Who is the event source?
 - Resize => window
 - Key presses => document
 - Load => document
 - Click => button, image,
 - Focus => input elements
 - Mouse => elements
- Details of the event object properties
(MouseEvent, KeyboardEvent,).

Refer to online API

```
function keyHandler(ev: KeyboardEvent): void {  
    // put code here  
}  
  
function clickHandler(ev:MouseEvent): void {  
    // put code here  
}  
  
document.addEventListener("keypress", keyHandler);  
const myLogo = document.getElementById("myLogo");  
myLogo.addEventListener("click", clickHandler);
```

addEventListener

inline event attributes

```
<button onclick="clickFunction()">Click Me</button>
```

```
function clickFunction() {  
    // put code here  
}
```

Practice: Event Handling

Counting Click



Given the HTML and JavaScript code in the above link, implement functionality for the `<h1 id="h1counter">` element so that when it is clicked, the `<p id="outputH1counter">` displays 'Clicked __ times'.